

Final Evolution Lab — UE5 Pixel Streaming Deployment Guide

Version: 1.0.0

Last Updated: April 2, 2026

Maintainer: deploy@finalevolutiongroup.com

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 3. [Prerequisites](#)
 4. [Cost Estimates](#)
 5. [Quick Start](#)
 6. [Step-by-Step Deployment](#)
 - [Phase 1: AWS Infrastructure \(Terraform\)](#)
 - [Phase 2: Build UE5 Project](#)
 - [Phase 3: Deploy to AWS](#)
 - [Phase 4: SSL & Domain Setup](#)
 - [Phase 5: Verify Deployment](#)
 7. [Scaling Guide](#)
 8. [Monitoring & Observability](#)
 9. [Security Best Practices](#)
 10. [Troubleshooting](#)
 11. [Backup & Disaster Recovery](#)
 12. [Cost Optimization](#)
-

Overview

Final Evolution Lab is a basketball training game built with **Unreal Engine 5** that uses **Pixel Streaming** to deliver cloud-based gameplay. Players connect through a web browser or iOS app, and the game renders on GPU-powered cloud servers, streaming the video output via WebRTC.

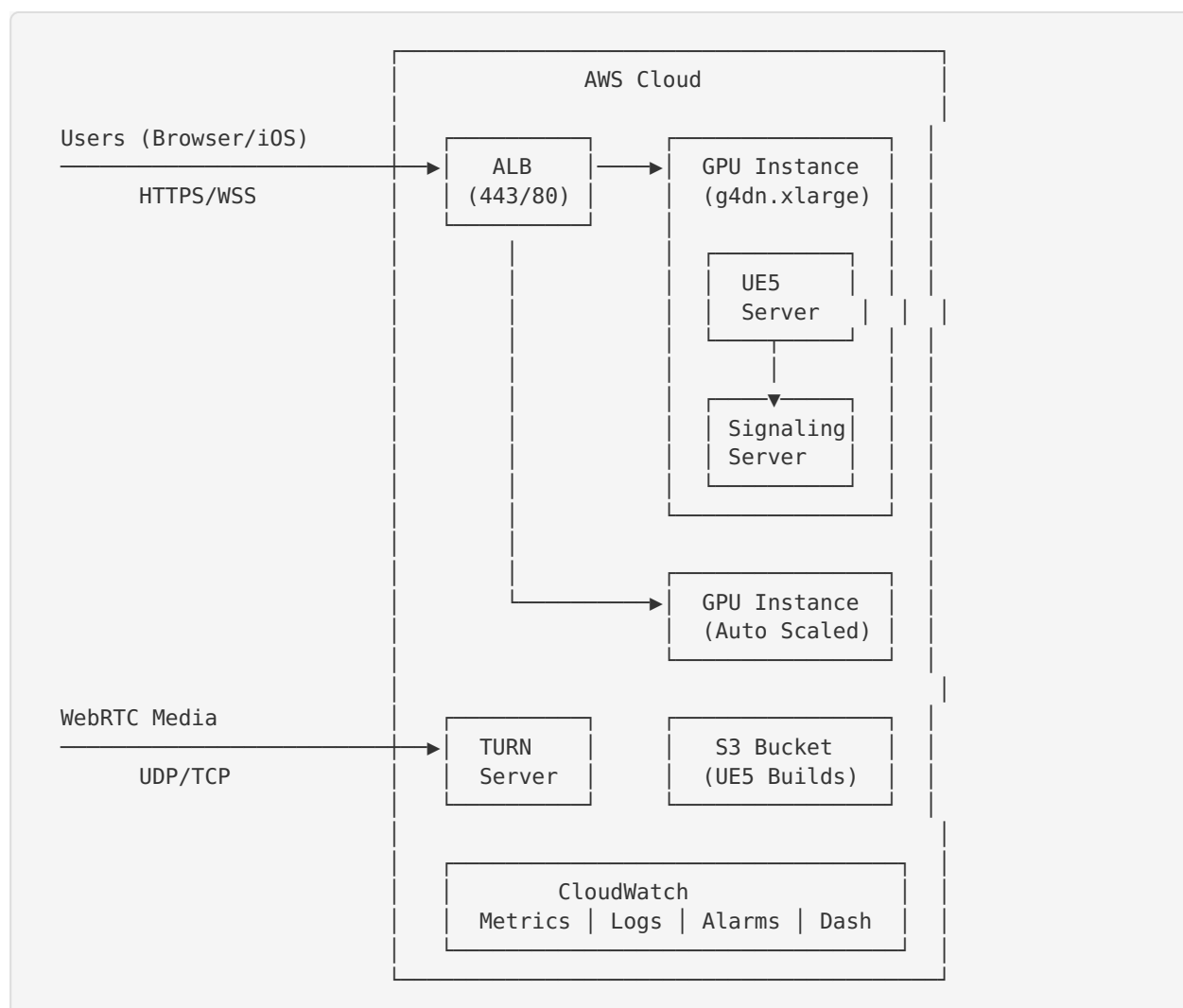
Key Features

- **17 Game Modes** (Basketball H2H, 3v3, 5v5, Karate, MMA, Boxing, Soccer, etc.)
- **23 Training Exercises** with AI-generated animations
- **OpenCap Body Scanning** for motion analysis
- **Live Link Face** integration
- **Family Sharing** and **Shard Economy** system
- **Cross-platform**: Web, iOS, future Android

Infrastructure Components

Component	Technology	Purpose
GPU Servers	EC2 g4dn.xlarge	UE5 rendering + Pixel Streaming
Signaling Server	Node.js + WebSocket	Player-streamer connection brokering
TURN/STUN Server	CoTURN	WebRTC NAT traversal relay
Load Balancer	AWS ALB	Traffic distribution + SSL termination
Build Storage	AWS S3	UE5 build artifact storage
Monitoring	CloudWatch	Metrics, logs, alarms, dashboards
Auto Scaling	ASG	Dynamic capacity based on demand

Architecture



Data Flow

1. **Player connects** via browser/iOS to `https://stream.finalevolutiongroup.com`
2. **ALB terminates SSL** and routes to a GPU instance
3. **Signaling server** (WebSocket) negotiates WebRTC connection
4. **UE5 server** renders the game and encodes video (H.264 via NVENC)
5. **WebRTC** streams encoded video/audio to the player's browser
6. **TURN server** relays media if direct peer connection fails (symmetric NAT)
7. **Player input** (mouse, keyboard, touch) is sent back via WebRTC data channel

Prerequisites

Required Accounts & Tools

- [] **AWS Account** with permissions for EC2, VPC, S3, IAM, CloudWatch, ACM, Route53
- [] **Terraform** >= 1.5.0 ([install guide](https://developer.hashicorp.com/terraform/install) (`https://developer.hashicorp.com/terraform/install`))
- [] **AWS CLI** v2 configured (`aws configure`)
- [] **Unreal Engine 5.4+** installed on a build machine with GPU

- [] **Domain name** with DNS managed by Route53 (or manual DNS)
- [] **Git** for version control

SSH Keys (Already Uploaded)

The following `.pem` key files are available for EC2 SSH access:

```
~/Uploads/FinalEvolutionLab.pem      # Primary SSH key
~/Uploads/Final Evolution Lab.pem    # Alternate SSH key (with space)
```

Important: Set correct permissions before use:

```
bash
chmod 400 ~/Uploads/FinalEvolutionLab.pem
chmod 400 ~/Uploads/"Final Evolution Lab.pem"
```

SSH Connection

```
# Connect to any GPU instance
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<INSTANCE_PUBLIC_IP>

# Connect using the alternate key
ssh -i ~/Uploads/"Final Evolution Lab.pem" ubuntu@<INSTANCE_PUBLIC_IP>
```

System Requirements (Build Machine)

Resource	Minimum	Recommended
CPU	8 cores	16+ cores
RAM	32 GB	64+ GB
Disk	200 GB SSD	500+ GB NVMe
GPU	NVIDIA GTX 1080	RTX 3080+
OS	Ubuntu 22.04	Ubuntu 22.04

System Requirements (GPU Server / EC2)

Resource	g4dn.xlarge	g4dn.2xlarge	g5.xlarge
vCPU	4	8	4
RAM	16 GB	32 GB	16 GB
GPU	T4 (16GB)	T4 (16GB)	A10G (24GB)
GPU VRAM	16 GB	16 GB	24 GB
Network	Up to 25 Gbps	Up to 25 Gbps	Up to 25 Gbps
On-Demand \$/hr	\$0.526	\$0.752	\$1.006
Spot \$/hr (est.)	~\$0.16	~\$0.23	~\$0.30

Cost Estimates

Small Scale (1-5 concurrent users)

Resource	Monthly Cost
1x g4dn.xlarge (on-demand 12h/day)	~\$190
1x ALB	~\$25
1x c5.xlarge TURN server	~\$50
S3 (10GB builds)	~\$0.25
CloudWatch	~\$10
NAT Gateway	~\$35
Total	~\$310/month

Medium Scale (5-20 concurrent users)

Resource	Monthly Cost
2-5x g4dn.xlarge (on-demand)	~\$380-950
1x ALB	~\$30
1x c5.xlarge TURN	~\$50
S3 + Data Transfer	~\$15
CloudWatch	~\$20
NAT Gateway	~\$50
Total	~\$545-1,115/month

Large Scale (20-100 concurrent users)

Resource	Monthly Cost
5-25x g4dn.xlarge (mixed on-demand/spot)	~\$600-3,000
1x ALB	~\$50
2x c5.xlarge TURN	~\$100
S3 + Transfer	~\$50
CloudWatch	~\$40
NAT Gateway	~\$80
Total	~\$920-3,320/month

Cost Optimization Tip: Use Spot Instances for up to 70% savings. See [Cost Optimization](#).

Quick Start

```
# 1. Clone and navigate to deployment
cd /home/ubuntu/rork-final-evolution-lab/deployment

# 2. Configure Terraform variables
cp aws/terraform.tfvars.example aws/terraform.tfvars
# Edit terraform.tfvars with your values

# 3. Deploy infrastructure
cd aws
terraform init
terraform plan
terraform apply

# 4. Build and upload UE5 project (on build machine with GPU)
cd ../scripts
./build-and-upload.sh --config Shipping

# 5. Deploy to instances
./deploy-fleet.sh

# 6. Setup SSL
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<INSTANCE_IP>
sudo /opt/fel/scripts/setup-ssl.sh stream.finalevolutiongroup.com
```

Step-by-Step Deployment

Phase 1: AWS Infrastructure

1.1 Configure Terraform

```
cd /home/ubuntu/rork-final-evolution-lab/deployment/aws

# Copy and edit variables
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars` :

```
project_name      = "final-evolution-lab"
environment       = "production"
aws_region        = "us-west-2"
gpu_instance_type = "g4dn.xlarge"
key_pair_name     = "FinalEvolutionLab" # Must match your AWS key pair name
asg_min_size      = 1
asg_max_size      = 10
asg_desired_capacity = 2
domain_name       = "stream.finalevolutiongroup.com"
notification_email = "deploy@finalevolutiongroup.com"
```

1.2 Import SSH Key to AWS

Before deploying, ensure your `.pem` key pair is registered in AWS:

```
# If creating a new key pair, import the public key
aws ec2 import-key-pair \
  --key-name "FinalEvolutionLab" \
  --public-key-material fileb:// ~/.ssh/FinalEvolutionLab.pub \
  --region us-west-2
```

If you already created the key pair in the AWS Console and downloaded the `.pem` file (which matches the uploaded `FinalEvolutionLab.pem`), no import is needed.

1.3 Deploy Infrastructure

```
# Initialize Terraform
terraform init

# Preview changes
terraform plan -out=plan.tfplan

# Apply (creates all AWS resources)
terraform apply plan.tfplan

# Save outputs for later use
terraform output -json > ../terraform-outputs.json
```

1.4 Verify Infrastructure

```
# Check ALB is created
terraform output alb_dns_name

# Check S3 bucket
terraform output s3_builds_bucket

# Check TURN server IP
terraform output turn_server_ip

# Get SSH command
terraform output ssh_command
```

Phase 2: Build UE5 Project

Note: This must be done on a machine with UE5 installed and a GPU.

2.1 Ensure Pixel Streaming is Enabled

Open `UnrealStarter/BasketballGame/BasketballGame.uproject` and verify:

```
{
  "Plugins": [
    { "Name": "PixelStreaming", "Enabled": true },
    { "Name": "PixelStreamingPlayer", "Enabled": true }
  ]
}
```

2.2 Configure Pixel Streaming Settings

Create/edit `Config/DefaultEngine.ini` :

```
[PixelStreaming]
AllowPixelStreamingCommands=true
WebRTCFps=60
WebRTCStartBitrate=10000000
WebRTCMaxBitrate=20000000
WebRTCMinBitrate=5000000
```

```
[/Script/Engine.RendererSettings]
r.DefaultFeature.AntiAliasing=2
r.ScreenPercentage=100
```

2.3 Package the Build

```
# Using the automated script
cd /home/ubuntu/rork-final-evolution-lab/deployment/scripts
./build-and-upload.sh --config Shipping

# Or manually with RunUAT
$RUN_UAT BuildCookRun \
  -project="/path/to/BasketballGame.uproject" \
  -platform=Linux \
  -serverconfig=Shipping \
  -cook -build -stage -pak -archive \
  -archivedirectory="./Builds/LinuxServer" \
  -server -noclient -compressed
```

2.4 Verify the Build

```
./verify-build.sh ./Builds/LinuxServer/
```

Phase 3: Deploy to AWS

3.1 Upload Build to S3

```
# If not already uploaded by build-and-upload.sh
aws s3 cp Builds/fel-build-*.tar.gz s3://$(terraform output -raw s3_builds_bucket)/
builds/
```

3.2 Deploy to Running Instances

Single instance (manual):

```
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<INSTANCE_IP>
sudo /opt/fel/scripts/deploy.sh --version <BUILD_VERSION>
```

Fleet-wide (rolling deployment):

```
./deploy-fleet.sh --version <BUILD_VERSION>
```

3.3 Verify Deployment

```
# On the instance
./health-check.sh http://localhost:8888

# Remotely
curl -s https://stream.finalevolutiongroup.com/healthz | jq .
```

Phase 4: SSL & Domain Setup

4.1 Configure DNS

Point your domain to the ALB:

```
# If using Route53 (automated by Terraform)
terraform output alb_dns_name
# Create a CNAME or A record pointing to this DNS name

# Or manually in your DNS provider:
# stream.finalevolutiongroup.com -> CNAME -> <ALB_DNS_NAME>
```

4.2 SSL Certificate

SSL is handled at two levels:

1. **ALB Level** (recommended): ACM certificate auto-created by Terraform
2. **Instance Level** (optional): Let's Encrypt via certbot

```
# ACM certificate is created automatically by Terraform
# For DNS validation, add the CNAME records shown in terraform output

# For Let's Encrypt on individual instances (direct access):
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<INSTANCE_IP>
sudo ./setup-ssl.sh stream.finalevolutiongroup.com
```

Phase 5: Verify Deployment

```
# Run the comprehensive health check
./health-check.sh https://stream.finalevolutiongroup.com --verbose

# Test WebSocket connectivity
python3 -c "
import asyncio, websockets
async def test():
    async with websockets.connect('wss://stream.finalevolutiongroup.com/ws') as ws:
        await ws.send('{\"type\": \"player\"}')
        resp = await ws.recv()
        print(f'Response: {resp}')
asyncio.run(test())
"

# Test game modes API
curl -s https://stream.finalevolutiongroup.com/api/modes | jq .
```

Scaling Guide

Horizontal Scaling (More Instances)

Each GPU instance supports **1-3 concurrent streams** depending on game complexity.

```
# Manual scaling
aws autoscaling set-desired-capacity \
  --auto-scaling-group-name "final-evolution-lab-production-gpu-asg" \
  --desired-capacity 5 \
  --region us-west-2

# Update Terraform for persistent change
# In terraform.tfvars:
asg_desired_capacity = 5
asg_max_size = 20
```

Vertical Scaling (Bigger Instances)

Upgrade Path	Streams/Instance	Use Case
g4dn.xlarge → g4dn.2xlarge	2-4	More CPU headroom
g4dn.xlarge → g5.xlarge	3-5	Better GPU (A10G)
g4dn.xlarge → g5.2xlarge	4-6	Maximum performance

Auto Scaling Triggers

The default configuration scales based on:

- **CPU utilization**: Scale up at 70%, scale down at 30%
- **ALB request count**: Target 2 concurrent streams per instance
- **Scheduled**: Scale down at 10 PM PST, up at 6 AM PST

Multi-Region Deployment

For global low-latency:

1. Deploy infrastructure in multiple regions (us-west-2, eu-west-1, ap-northeast-1)
2. Use Route53 latency-based routing
3. Each region has independent ASG and TURN servers

Monitoring & Observability

CloudWatch Dashboard

Access: <https://us-west-2.console.aws.amazon.com/cloudwatch/home?region=us-west-2#dashboards:name=final-evolution-lab-production-pixel-streaming>

Tracks:

- CPU utilization across GPU fleet
- ASG capacity (in-service, desired, total)

- ALB request count and latency
- HTTP response codes (2xx, 4xx, 5xx)
- Target group health

Custom GPU Metrics

The `monitor-gpu.sh` script publishes to CloudWatch every minute:

- GPU utilization (%)
- GPU memory used (MB) and percentage
- GPU temperature (°C)
- GPU power draw (W)
- Active streaming sessions (count)

```
# Install cron job on each instance
echo "* * * * * /opt/fel/scripts/monitor-gpu.sh" | crontab -
```

Alarms

Alarm	Threshold	Action
High CPU	>85% for 15min	SNS email notification
Unhealthy hosts	>0	SNS email notification
ALB 5XX errors	>10 in 5min	SNS email notification
High latency	>5s avg	SNS email notification
ASG at max capacity	= max_size	SNS email notification

Log Access

```
# View Pixel Streaming logs
aws logs tail /fel/pixel-streaming --follow --region us-west-2

# View UE5 server logs
aws logs tail /fel/ue5-server --follow --region us-west-2

# View signaling server logs
aws logs tail /fel/signaling-server --follow --region us-west-2

# On the instance directly
tail -f /opt/fel/logs/ue5-server.log
tail -f /opt/fel/logs/signaling.log
```

Security Best Practices

Network Security

- [x] VPC with public/private subnets
- [x] Security groups with least-privilege rules

- ☒ VPC Flow Logs enabled for audit
- ☒ NAT Gateway for private subnet outbound access
- ☐ Restrict SSH to specific IP ranges (update `security_groups.tf`)
- ☐ Enable AWS WAF on ALB for DDoS protection

Instance Security

- ☒ IMDSv2 required (metadata service hardened)
- ☒ EBS volumes encrypted at rest
- ☒ SSM access for secure remote management
- ☒ Log rotation configured
- ☐ Enable SELinux or AppArmor
- ☐ Install and configure fail2ban

Data Security

- ☒ S3 buckets with public access blocked
- ☒ S3 server-side encryption (AES-256)
- ☒ S3 versioning enabled for build rollback
- ☒ ALB access logs stored in separate bucket
- ☒ TLS 1.3 enforced on ALB

Credential Management

- ☒ TURN secret auto-generated if not provided
- ☒ IAM roles (no long-lived access keys on instances)
- ☒ Terraform sensitive variables marked
- ☐ Use AWS Secrets Manager for TURN secret
- ☐ Rotate SSH keys periodically

SSH Key Security

```
# Always set correct permissions
chmod 400 ~/Uploads/FinalEvolutionLab.pem
chmod 400 ~/Uploads/"Final Evolution Lab.pem"

# Never commit .pem files to git
# .pem files are in .gitignore

# Consider using SSM Session Manager instead of SSH
aws ssm start-session --target <INSTANCE_ID> --region us-west-2
```

Troubleshooting

Common Issues

GPU Not Detected After Instance Launch

```
# Check if GPU is present
lspci | grep -i nvidia

# Check driver installation
nvidia-smi

# If drivers failed, reinstall
sudo apt-get install -y nvidia-driver-535
sudo reboot
```

UE5 Server Won't Start

```
# Check logs
journalctl -u fel-ue5-server --no-pager -n 50

# Check if binary exists and is executable
ls -la /opt/fel/current/FinalEvolutionLab.sh

# Check GPU memory
nvidia-smi

# Try running manually
cd /opt/fel/current
./FinalEvolutionLab.sh -RenderOffscreen -log 2>&1 | head -50
```

WebSocket Connection Fails

```
# Check signaling server
curl http://localhost:8888/healthz

# Check WebSocket port
netstat -tlnp | grep 8889

# Test WebSocket directly
websocat ws://localhost:8889
```

High Latency / Poor Quality

```
# Check GPU utilization
nvidia-smi

# If GPU > 90%, reduce resolution:
# Edit launch parameters: -ResX=1280 -ResY=720

# Check network bandwidth
iperf3 -c <client-ip>

# Verify TURN server is working (for NAT traversal)
turnutils_uclient -T -u test -w test <TURN_IP>
```

Instances Failing Health Checks

```
# Check all services
systemctl status fel-signaling fel-ue5-server nginx

# Run full health check
/opt/fel/scripts/health-check.sh --verbose

# Check ALB target group health
aws elbv2 describe-target-health \
  --target-group-arn <TG_ARN> \
  --region us-west-2
```

Build Download Fails from S3

```
# Check IAM role
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/

# Test S3 access
aws s3 ls s3://<BUCKET>/builds/ --region us-west-2

# Check instance profile
aws sts get-caller-identity
```

TURN Server Not Working

```
# Check coturn status
sudo systemctl status coturn

# Check logs
sudo tail -f /var/log/turnserver.log

# Test TURN connectivity
turnutils_uclient -e <PUBLIC_IP> -p 3478 -T
```

Emergency Procedures

Rollback Deployment

```
# Single instance
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<IP>
sudo /opt/fel/scripts/deploy.sh --rollback

# Fleet-wide: Cancel instance refresh
aws autoscaling cancel-instance-refresh \
  --auto-scaling-group-name "final-evolution-lab-production-gpu-asg" \
  --region us-west-2
```

Scale to Zero (Cost Saving)

```
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name "final-evolution-lab-production-gpu-asg" \
  --min-size 0 --desired-capacity 0 \
  --region us-west-2
```

Destroy All Infrastructure

```
cd /home/ubuntu/rork-final-evolution-lab/deployment/aws
terraform destroy
```

Backup & Disaster Recovery

What to Back Up

Item	Location	Backup Method	Frequency
UE5 Builds	S3	S3 versioning + cross-region replication	Every build
Terraform State	S3 backend	S3 versioning	Every apply
Game Config	Git repo	Git push	Every change
CloudWatch Logs	CloudWatch	30-day retention (configurable)	Continuous
TURN Server Config	/etc/turnserver.conf	AMI snapshot	Weekly

Recovery Procedures

Rebuild from Scratch

```
# 1. Deploy infrastructure
cd deployment/aws && terraform apply

# 2. Deploy latest build
cd ../scripts && ./deploy-fleet.sh
```

Recovery Time: ~30 minutes

Replace Single Instance

The ASG automatically replaces failed instances. Manual:

```
aws autoscaling terminate-instance-in-auto-scaling-group \
  --instance-id <INSTANCE_ID> \
  --should-decrement-desired-capacity false \
  --region us-west-2
```

Recovery Time: ~10 minutes

Cost Optimization

1. Spot Instances (Up to 70% Savings)

Add to `ec2.tf` launch template:

```
instance_market_options {
  market_type = "spot"
  spot_options {
    max_price           = "0.20" # Max bid
    spot_instance_type = "one-time"
    instance_interruption_behavior = "terminate"
  }
}
```

2. Scheduled Scaling

Already configured:

- Scale down to minimum at 10 PM PST (low traffic)
- Scale up at 6 AM PST (peak hours)

3. Reserved Instances

For baseline capacity:

- 1-year RI: ~40% savings vs on-demand
- 3-year RI: ~60% savings vs on-demand

4. Right-Sizing

- Start with g4dn.xlarge (most cost-effective GPU)
- Monitor GPU utilization; upgrade only if > 80% sustained
- Consider g5.xlarge only if T4 GPU is insufficient

5. S3 Lifecycle Rules

Already configured:

- Standard → Standard-IA after 30 days
- Standard-IA → Glacier after 90 days
- Delete after 365 days

6. NAT Gateway Optimization

- Consider VPC endpoints for S3 (eliminates NAT charges for S3 traffic)
- Add to Terraform:

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.${var.aws_region}.s3"
  route_table_ids = [aws_route_table.private.id]
}
```

Appendix

File Structure

```

deployment/
├── aws/
│   ├── main.tf
│   ├── versions.tf
│   ├── variables.tf
│   ├── vpc.tf
│   ├── security_groups.tf
│   ├── iam.tf
│   ├── s3.tf
│   ├── alb.tf
│   ├── ec2.tf
│   ├── cloudwatch.tf
│   ├── outputs.tf
│   ├── terraform.tfvars.example
│   └── templates/
│       ├── user_data.sh.tpl
│       └── turn_user_data.sh.tpl
├── scripts/
│   ├── setup-gpu-server.sh
│   ├── setup-ssl.sh
│   ├── setup-coturn.sh
│   ├── build-and-upload.sh
│   ├── verify-build.sh
│   ├── deploy.sh
│   ├── deploy-fleet.sh
│   ├── health-check.sh
│   └── monitor-gpu.sh
└── docs/
    └── UE5_DEPLOYMENT_GUIDE.md

```

Terraform infrastructure
 # Core configuration & data sources
 # Provider versions & backend
 # All configurable variables
 # VPC, subnets, routing, flow logs
 # ALB, GPU, TURN security groups
 # Roles & policies (instance, deploy)
 # Build artifact storage
 # Load balancer, target groups, certs
 # Launch template, ASG, TURN server
 # Logs, alarms, dashboard
 # Deployment outputs
 # Example variable values
 # GPU instance bootstrap
 # TURN server bootstrap
 # Operational scripts
 # Manual GPU server setup
 # Let's Encrypt SSL setup
 # Dedicated TURN server setup
 # Build UE5 + upload to S3
 # Build verification checks
 # Single-instance deployment
 # Fleet-wide rolling deployment
 # Comprehensive health checks
 # GPU metrics → CloudWatch
 # This guide

Useful Commands

```

# SSH to instance
ssh -i ~/Uploads/FinalEvolutionLab.pem ubuntu@<IP>

# Check all services
sudo systemctl status fel-signaling fel-ue5-server nginx coturn

# View live GPU stats
watch -n1 nvidia-smi

# View streaming sessions
curl -s localhost:8888/healthz | jq .

# Restart everything
sudo systemctl restart fel-signaling fel-ue5-server nginx

# View deployment info
cat /opt/fel/config/current-deployment.json

# Tail all logs
tail -f /opt/fel/logs/*.log

```

Environment Variables

These are set in `/etc/fel-env` on each instance:

Variable	Description
----- -----	
<code>AWS_REGION</code>	AWS region
<code>S3_BUILDS_BUCKET</code>	S3 bucket for builds
<code>ENVIRONMENT</code>	dev/staging/production
<code>TURN_SERVER_IP</code>	TURN server private IP
<code>TURN_SECRET</code>	TURN authentication secret
<code>SIGNALING_PORT</code>	Signaling HTTP port (8888)
<code>WEBSOCKET_PORT</code>	WebSocket port (8889)
<code>LOG_GROUP</code>	CloudWatch log group

For additional support, contact deploy@finalevolutiongroup.com